BERTHAUD Yanis 17.04.2026

PROJET DE SPECIALITE

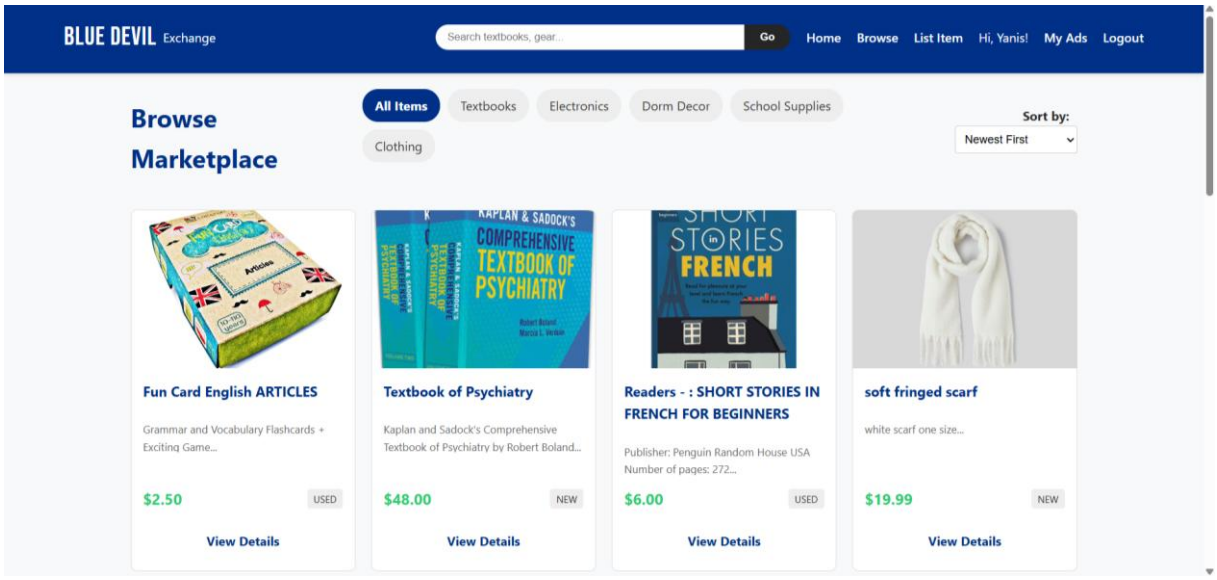
My Digital School Année 2025.2026

I – Présentation du Projet :

Blue Devil Exchange est une application web dynamique conçue spécifiquement pour l'écosystème de Duke University. Elle permet aux étudiants et au personnel administratif de l'université de mettre en vente, d'acheter ou d'èchanger du matériel (manuels scolaires, équipement de dortoirs, électronique) au sein d'un environnement numérique sécurisé et localisé.

- **Auteur :** Yanis BERTHAUD
- **Date de Rendu :** 17.04.2026
- **Etablissement :** Duke University
- **Statut du Projet :** Déployé en environnement de production (Version 1.0).
- **URL de production :** http://blue-devil-exchange.infinityfree.me/index.php

Aperçu du système :



Résumé Technique :

- **Langage :** PHP (Programmation Orientée Objet)
- **Base de données :** MySQL via l'interface PDO.
- **Hébergement :** Serveur Linux (LAMP) avec accès FTP.

II – Analyse du besoin et Etude de faisabilité :

Analyse de la problématique :

Dans un environnement universitaire de l'envergure de Duke University, les étudiants sont confrontés à un renouvellement constant de leurs besoins matériels : manuels scolaires (textbooks) spécifiques à chaque semestre, fournitures de bureau, ou encore aménagements des dortoirs (dorm decord).

Cependant, il manque une solution centralisée pour faciliter l'économie circulaire au sein du campus. Les solutions actuelles présentent plusieurs limites :

- **Manque de sécurité :** Les plateformes grand public ne garantissent pas que l'interlocuteur appartient à la communauté universitaire.
- **Complexité logistique :** Les frais de port et les délais de livraison découragent l'achat d'occasion entre étudiant.
- **Visibilité réduite :** Les annonces se perdent souvent sur des groupes de réseaux sociaux non structurés.
-

La solution : Blue Devil Exchange :

L'objectif est de créer un Marketplace de proximité, sécurisé et structuré, permettant une mise en relation directe. La plateforme se concentre sur trois piliers :

1. **Confidentialité :** Un accès restreint aux membres inscrits.
2. **Simplicité :** Un système de catégories (Electronics, Textbooks, Clothins, etc) pour une recherche instantanée.
3. **Localisation :** Les échanges se font physiquement sur le campus, éliminant les frais d'envoi.

Public Cible :

Pour concevoir l'interface, deux profils types ont été identifiés :

- **L'étudiant vendeur :** Souhaite libérer de l'espace dans son dortoir en fin de semestre et récupérer un budget pour ses prochains achats.
- **L'étudiant acheteur :** Recherche du matériel de qualité (ex : une calculatrice TI-84 ou un micro-ondes) à un prix réduit par rapport au neuf.

Étude de faisabilité technique :

Le projet repose sur une infrastructure LAMP (Linux, Apache, MySQL, PHP), choisie pour sa robustesse et sa large compatibilité avec les hébergeurs web.

- PHP (POO) : Pour une logique métier modulaire et évolutive.
- MySQL (PDO) : Pour une gestion sécurisée et performante des données.

III – Architecture et Conception Technique :

Modélisation de la Base de Données :

Le système repose sur une base de données relationnelles MySQL. La structure a été pensée pour être évolutive. Elle s'articule autour de trois entités principales :

1. **Users :** Stocke les informations d'authentification (ID, email, mot de passe haché).
2. **Catégories :** Liste prédéfinir des types d'articles (Electronics, Textbooks, etc.) pour normaliser le filtrage.
3. **Listings** : Table centrale contenant les annonces. Chaque annonce est liée à un **user_id** (propriètaire) et un **category_id** (classification).

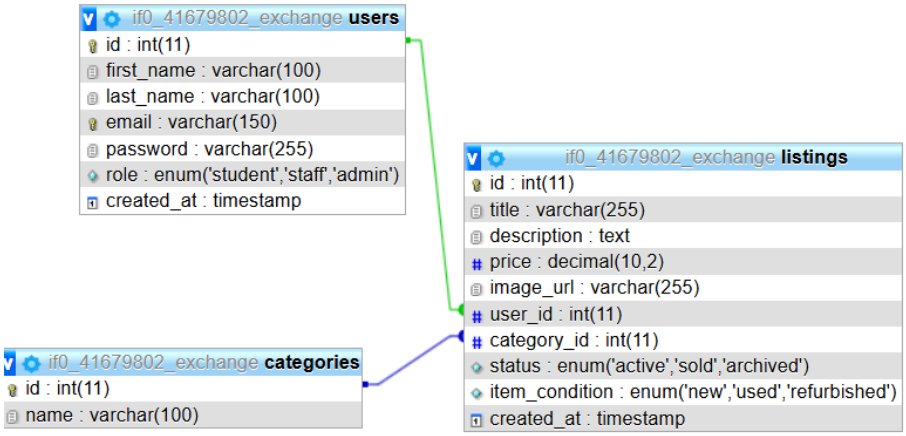
Note Technique : L'utilisation des clés étrangères assure l'intégrité référentielle des données 'par exemple, la suppression d'un compte utilisateur va entraîner la suppression des annonces qui y sont associés).

Accès aux Données via PDO :

Au lieu d'utiliser les fonctions MySQL classiques, j'ai implenté l'interface PDO (PHP Data Objetcs).

- Sécurité : PDO permet l'utilisation de requêtes préparées, ce qui neutralise nativement les tentatives d'injections SQL.
- Abstraction : Cela rend le code plus flexible si l'on décidait de changer de système de gestion de base de données à l'avenir.
- Gestion des erreurs : L'utilisation du mode **PDO ::ERRMODE_EXCEPTION** permet de capturer les erreurs de connexion de manière propre via des blocs **try/catch.**

Le Schéma SQL :



IV – Développement des Fonctionnalités Clés :

Gestion de l'Authentification et des Sessions

L'une des premières briques développées a été le système d'inscription et de connexion. Pour garantir une expérience utilisateur fluide sur le campus :

- **Persistance :** L'utilisation de session_start() permet de maintenant l'utilisateur connecté durant toute sa navigation.
- **Sécurité des données :** Les mots de passe ne sont jamais stockés en clair. J'ai utilisé une fonction password_hash() pour le stockage et password_verify() pour l'authentification.
- **Redirection intelligente** : un utilisateur non connecté est automatiquement redirigé vers la page de login s'il tente de poster une annonce garantissant que chaque article est lié à un membre vérifié.

Le « Core » du Marketplace : le CRUD

Le projet repose sur le cycle CRUD (Create, Read, Udpate, Delete) pour la table des annonces :

- **Create :** Un formulaire complet (list_item.php) permet d'ajouter un titre, une description, un prix, une catégorie et un état (New, Used, Refurbished).
- **Read :** La page browse.php récupère dynamiquement les données pour afficher les « cards » d'articles.
- **Update / Delete :** Un espace « My Ads » permet à l'étudiant de modifier ses informations ou de supprimer un article une fois celui-ci vendu.
-

Gestion des Médias (Upload d'images)

Pour rendre la marketplace attractive, j'ai implémenté un système d'upload d'images :

- **Traitement :** Les images sont réceptionnées via la superglobale $_FILES, vérifiées, puis déplacées vers le répertoire /uploads.
- **Unicité :** Pour éviter que deux fichiers ne portent le même nom, chaque image est renommée dynamiquement avant d'être enregistrée.

Moteur de Recherche et Filtrage

Afin de faciliter la navigation au sein des 30 articles de test (et plus à l'avenir), j'ai développé :

- **Recherche textuelle :** Une barre de recherche filtrant les résultats via l'opérateur SQL LIKE.

- **Tri par catégories :** Des filtres dynamiques permettant d'isoler, par exemple, uniquement les « Textbooks » ou les « Electronics ».

V – Sécurité Applicative et Intégrité des Données

Protection contre les Injections SQL.

La menace principale pour une application gérant une base de données est l'injection SQL. Pour neutraliser ce risque, j'ai banni les requêtes brutes au profit de l'interface PDO.

- **Requêtes préparées** : Toutes les interactions avec la table listings et users utilisent des marqueurs nommés (ex : user_id).
- **Typage des données :** Le moteur PDO se charge d'assainir les données envoyées par les utilisateurs avant qu'elles n'atteignent le serveur SQL, rendant impossible l'exécution de code malveillant via les formulaires.

Sécurisation de l'Authentification.

La sécurité des utilisateurs de Duke University est une priorité. J'ai mis en place deux barrières majeures :

- **Hachage Argon2/Bcrypt :** Via la fonction password_hash(), les mots de passe sont transformés en empreintes cryptographiques irréversibles. Même en cas d'accès non autorisé à la base de données, les comptes restent protégés.
- **Protection des Sessions :** A chaque connexion, un identifiant de session unique est généré. Les pages sensibles (comme la création d'annonce) vérifient systématiquement l'existence de $_SESSION['user_id'] avant de charger le contenu.

Sécurisation du Système de Fichiers

L'upload d'images est un vecteur d'attaque courant. Pour sécuriser le dossier /uploads, j'ai appliqué les règles suivantes :

- **Vérification de l'extension :** Seuls les formats d'images standards (JPG, PNG, WEBP) sont acceptés.
- **Obfuscation des noms :** Chaque fichier est renommé avec un identifiant unique (uniqid()) pour empêcher l'éxécution de scripts cachés sous des noms de fichiers trompeurs.

VI – Déploiement et Mise en Production

Stratégie d'Hébergement

Le passage du développement local (XAMPP) à la production a nécessité une analyse des solutions disponibles.

- **Choix de l'infrastructure :** Initialement prévu sur un VPS, j'ai finalement opté pour un hébergement mutualisé LAMP via InfinityFree. Ce choix garantit une haute disponibilité et une configuration serveur optimisée pour PHP 8, tout en offrant les outils nécessaires (FTP, MySQL distant) pour un projet de type Marketplace.
- **Gestion des fichiers :** Le transfert des données a été réalisé via le protocole SFTP avec le client FileZilla, permettant un déploiement granulaire et une mise à jour rapide des scripts en cas de correctif.

Configuration du Serveur Apache (.htaccess).

J'ai configuré un fichier /htaccess à la racine du serveur

- **Redirection DocumentRoot** : Le fichier redirige automatiquement tout le trafic vers le dossier /public. Cela permet de masquer les dossiers sensibles (/config, /src) aux yeux des utilisateurs externes, renforçant ainsi la sécurité du code source.

Migration et Synchronisation de la Base de Données

Le déploiement de la base de données s'est fait en deux étapes :

1. **Exportation :** Génération d'un dump SQL depuis l'environnement local.
2. **Importation et Adaptation :** Migration vers le serveur MySQL distant. Les scripts ont été adaptés pour prendre en compte les contraintes du serveur de production (préfixes de tables, droits d'utilisateurs restreints).

Gestion des Incidents de Déploiement

Le déploiement a présenté des défis techniques que j'ai dû résoudre en temps réel :

- **Résolution du Socket MySQL :** Correction de l'erreur SQLSTATE[2002] en basculant la connexion de localhost vers le hostname explicite du serveur distant.
- **Permissions de fichiers (CHMOD)** : Configuration des droits d'écriture sur le dossier /uploads (passage en 777) pour autoriser le serveur Web à enregistrer les images des annonces postées par les étudiants.

- **Sensibilité à la casse :** Harmonisation des appels require_once pour respecter la casse des noms de dossiers (Linux étant sensible aux majuscules, contrairement à Windows).

VII – Tests, Recette et Validation du Jeu de Données

Stratégie de tests (Recette)

Pour valider la robustesse de l'application avant sa livraison finale, une batterie de tests a été effectuée directement sur l'environnement de production :

- **Tests de navigation :** vérification de l'intégrité des liens et de la fluidité du passage entre le Dashboard et le Marketplace.
- **Validation du CRUD :** Test d'insertion d'articles avec et sans photos pour vérifier la gestion des valeurs par défaut en base de données.

Injection d'un jeu de données réaliste (30 offres)

Afin de simuler une activité réelle sur le campus de Duke, j'ai injecté des offres dans la base de données d'une trentaine d'articles environ et assez variés. Ce test a permis de confirmer la pertinence du système de filtrage notamment.

Internationalisation (Le choix du $ USD)

Le projet étant destiné à Duke University, l'intégralité du jeu de données a été saisie en anglais avec des prix en dollars américains. Ce choix renforce l'immersion et prouve la capacité de l'application à s'adapter à un marché international spécifique.

Limites de la version actuelle (V1)

Bien que parfaitement fonctionnelle, la version actuelle est une « Preuve de Concept ». Les échanges se font par contact direct après consultation, mais le système ne gère pas encore l'interaction en temps réel ni la transaction financière de manière automatisée.

VIII – Bilan, Perspectives et Evolutions Futures

Bilan du Projet

Le développement de Blue Devil Exchange a été une expérience complète couvrant l'intégralité du cycle de vie d'un logiciel. Ce projet m'a permis de consolider des compétences clés :

- **Maîtrise du Back-End** : Implémentation d'une logique métier en PHP POO et sécurisation des accès aux données.
- **Compétences DevOps** : Gestion du déploiement réel sur un serveur distant, configuration de fichier .htaccess et gestion des droits serveurs.
- **Adaptabilité** : Capacité à résoudre des bugs critiques en situation de production (connexion BDD, permissions de fichiers)

Roadmap : Evolutions de la V2 (A venir)

Bien que la plateforme actuelle soit fonctionnelle pour la mise en relation, plusieurs fonctionnalités majeures sont prévues pour transformer ce prototype en une application de marché complète :

1. **Système de messagerie instantané :**

   Objectif : Permettre aux étudiants de Duke de négocier et de convenir d'un point de rendez-vous directement sur le site, sans avoir à échanger leurs numéros personnels ou emails privés.

   Technologie : Utilisation potentielle de WebSockets ou d'appels AJAX réguliers pour une communication en temps réel.

2. **Système de Paiement Intégré :**

   Objectif : Sécuriser la transaction financière pour éviter les échanges de liquide sur le campus.

   Technologie : Intégration de l'API Stripe ou PayPal. Cela permettrait également de mettre en place un système de « paiement séquestre » (l'argent est dévloqué au vendeur seulement quand l'acheteur confirme la réception de l'objet comme sur Vinted par exemple).

3. **Système de Notation et de Confiance**

   Objectif : Créer un environnement de confiance.

   Fonctionnalité : A près chaque vente, l'acheteur et le vendeur peuvent se laisser une note (sur 5 étoiles) et un commentaire qui resteront visibles sur les profils publics (comme sur AirBnb).

IX – Conclusion

Le projet Blue Devil Exchange répond à un besoin concret de la vie étudiante. Au-délà de l'aspect technique, il incarne une démarche d'économie circulaire et de solidarité communautaire. La réussite du déploiement sur InfinityFree prouve que la plateforme est prête à accueillir ses premiers utilisateurs et à évoluer vers les fonctionnalités complexes citées précédemment.